FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Ian Ingal
Section:	AN-43
Professor:	Ms. Tan
Date:	

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

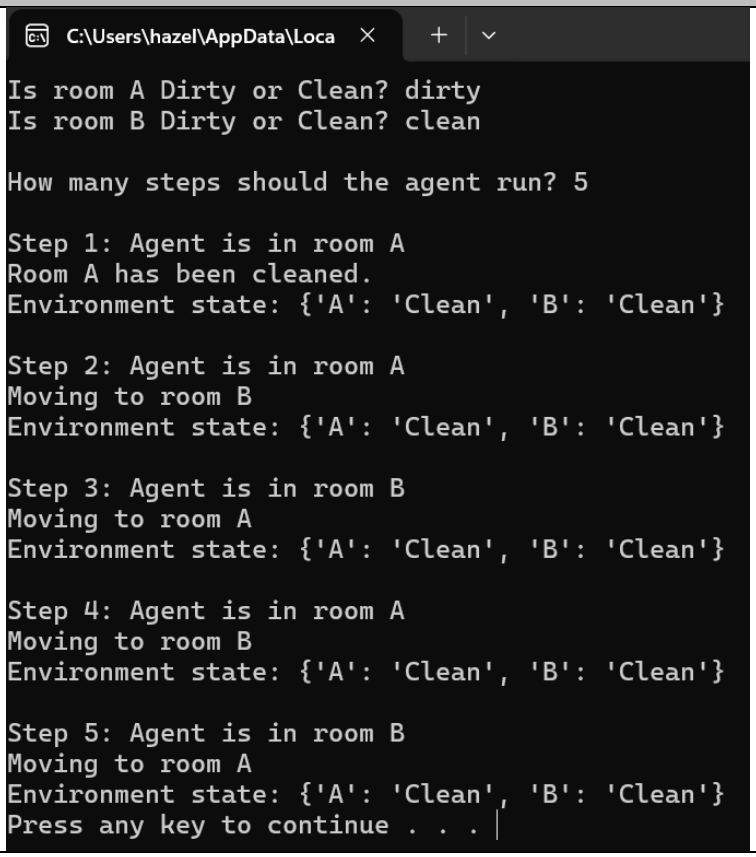
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

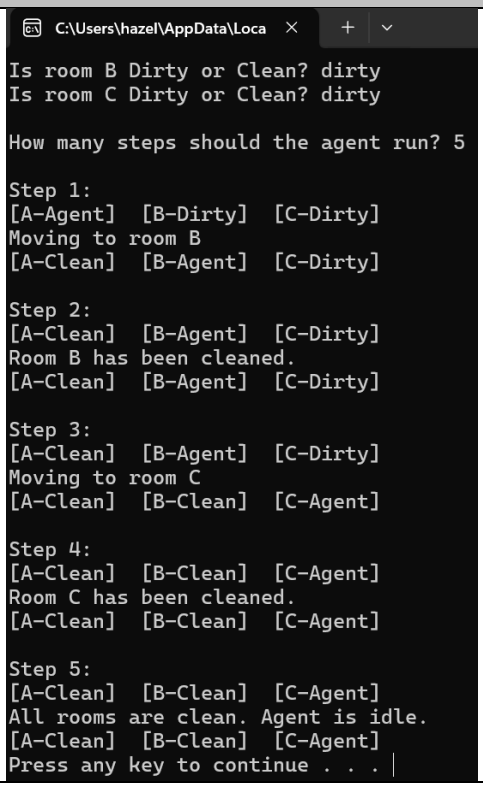
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
class Environment:
    def __init__(self):
        # Inpute dirty or clean
        print("Set initial state for the rooms (enter 'dirty' or 'clean'):")
        state_a = input("Is room A Clean or Dirty?: ").strip().lower()
        state_b = input("Is room B Clean or Dirty?: ").strip().lower()

        self.rooms = {
```

```
            'A': state_a if state_a in ['dirty', 'clean'] else 'dirty',
            'B': state_b if state_b in ['dirty', 'clean'] else 'dirty'
        }

    def is_dirty(self, room):
        return self.rooms[room] == 'dirty'

    def clean_room(self, room):
        self.rooms[room] = 'clean'

    def get_state(self):
        return self.rooms.copy()


class VacuumAgent:
    def __init__(self, start_room='A'):
        self.location = start_room  # Agent starting location

    def perceive_and_act(self, env):
        #If the current room is DIRTY, clean it
        if env.is_dirty(self.location):
            env.clean_room(self.location)
            return f"Cleaned room {self.location}"

        #If the current room is CLEAN, move to the other room
        else:
            self.move()
            return f"Moved to room {self.location}"

    def move(self):
        # Switch to room A and room B
        if self.location == 'A':
            self.location = 'B'
        else:
            self.location = 'A'




def run_simulation():
    env = Environment()

    # Ask user where the agent starts and how many steps to run
    start_loc = input("Enter starting room for agent ('A' or 'B'): ").strip().upper()
    if start_loc not in ['A', 'B']:
```

```
        start_loc = 'A'

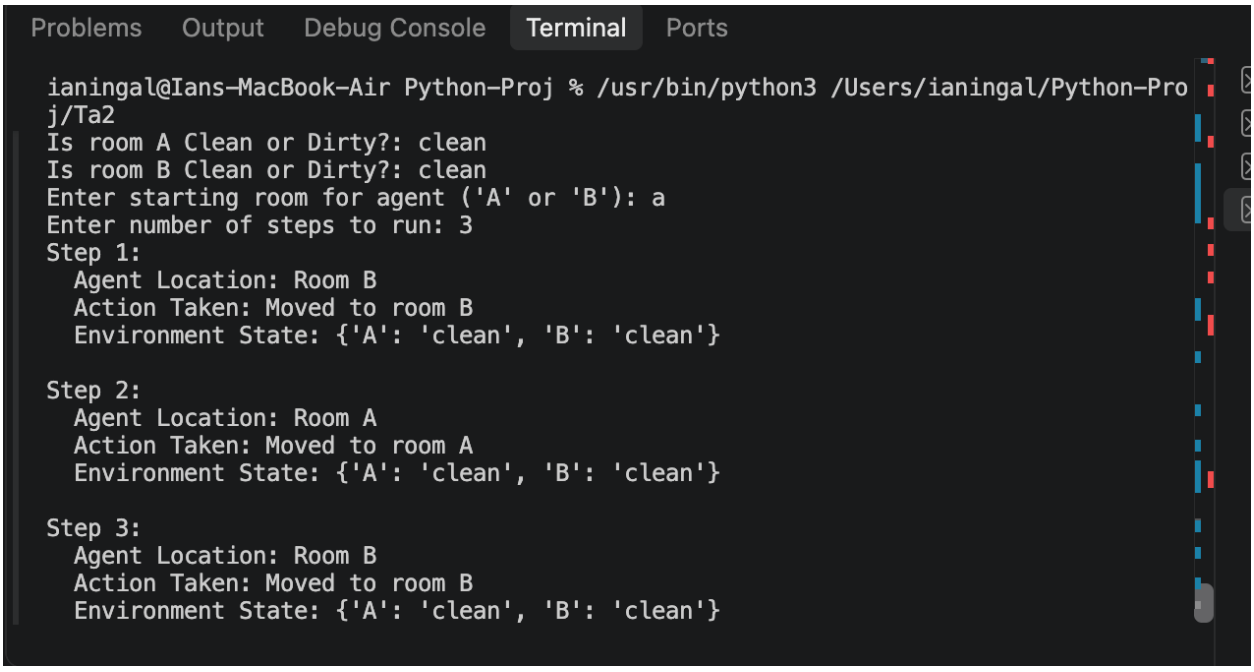
    steps = int(input("Enter number of steps to run: "))

    agent = VacuumAgent(start_loc)

    for step in range(1, steps + 1):
        action = agent.perceive_and_act(env)
        current_state = env.get_state()

        print(f"Step {step}:")
        print(f"  Agent Location: Room {agent.location}")
        print(f"  Action Taken: {action}")
        print(f"  Environment State: {current_state}\n")

if __name__ == "__main__":
    run_simulation()
```



Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
2. How does the agent decide when to clean and when to move?
3. What happens if both rooms are already clean at the start?
4. How would you modify the agent to handle **more than two rooms**?

Answer the Questions ⇩

1) If the room is dirty, it will clean it. And if the room is clean, it will move to other room.

2)It checks if the room is dirty, and clean it if it is. And if it is clean then it will move to another room.

3)The agent will check all the rooms to find dirty and it will keep moving to another room until it finish the steps to run.

4) Add extra rooms to the environment data and update the movement code so the agent can choose or cycle through multiple rooms instead of just two.